Introduction to Angular

Definition: Angular is an open-source web application development framework maintained by Google. It is used to build dynamic, interactive, and scalable web applications using TypeScript, HTML, and CSS.

1. Purpose of Angular: Angular helps developers build structured web applications with reusable components and organized code.

2. Component-Based Architecture: Angular applications are divided into components. Each component manages a specific part of the user interface.

3. TypeScript: Angular primarily uses TypeScript, which provides features such as static typing, classes, and interfaces.

4. Data Binding: Data binding connects component data with the user interface and helps display or update information.

5. Directives: Directives are used to modify the appearance or behavior of HTML elements and control how content is displayed.

6. Dependency Injection: Angular provides dependency injection to supply services and other required objects to components.

7. Routing: Angular routing allows users to navigate between different views or pages without reloading the entire application.

8. Reusability: Components and services can be reused across different parts of an application, reducing code duplication.

9. Applications: Angular is used to develop dashboards, e-commerce websites, enterprise applications, and single-page applications.

Conclusion: Angular is a powerful framework for developing modern web applications. Its component-based architecture, data binding, routing, and dependency injection make applications easier to develop and maintain.

AngularJS vs Angular

Definition: AngularJS is the original JavaScript-based framework released by Google in 2010, while Angular is its modern successor, introduced in 2016. Angular uses TypeScript and a component-based architecture to develop modern web applications.

Basis	AngularJS	Angular
1. Language	It is based on JavaScript.	It primarily uses TypeScript.
2. Release	It was released in 2010.	Angular 2 was released in 2016.
3. Architecture	It follows the MVC-based approach.	It follows a component-based architecture.
4. Components	It mainly uses controllers, scopes, and directives.	It uses components as the main building blocks.
5. Data Binding	It supports two-way data binding.	It supports both one-way and two-way data binding.
6. Performance	It may be slower for large applications due to its older architecture.	It offers improved performance and modern optimization features.
7. Mobile Support	It has limited support for modern mobile development.	It provides better support for modern mobile and responsive applications.
8. Dependency Injection	It supports dependency injection using its own mechanism.	It provides a more structured dependency injection system.
9. Command-Line Tool	It does not provide the modern Angular CLI.	It provides Angular CLI for creating, developing, and building projects.
10. Support	It is no longer officially supported by Google.	It is actively maintained by Google.

Conclusion: AngularJS is an older JavaScript-based framework, whereas Angular is a modern TypeScript-based framework with component-based architecture, improved tooling, and better support for building scalable applications.

MVC Framework

Definition: MVC stands for Model-View-Controller. It is a software architectural pattern that divides an application into three interconnected parts: Model, View, and Controller. This separation makes an application easier to develop, maintain, and test.

1. Model: The Model manages application data and business logic. It is responsible for handling data and interacting with the database when required.

2. View: The View represents the user interface. It displays data to users using elements such as text, forms, buttons, and tables.

3. Controller: The Controller handles user input and coordinates communication between the Model and the View. It processes user actions and determines the appropriate response.

4. Working of MVC: When a user performs an action, the Controller processes the input, communicates with the Model when necessary, and selects the appropriate View to display the result.

5. Separation of Concerns: MVC separates data handling, user interface, and application control into different parts, making the code more organized.

6. Reusability: Different parts of an application can be developed and maintained separately, allowing code to be reused where appropriate.

7. Easy Maintenance: Changes made to one part of the application generally have less impact on the other parts because responsibilities are separated.

8. Testing: Each part can be tested independently, which helps developers identify errors and improve application quality.

Example: In a student management application, the Model manages student details, the View displays student information, and the Controller handles actions such as adding or updating a student.

Conclusion: MVC is an architectural pattern that organizes an application into Model, View, and Controller. It improves code organization, maintainability, and testing. AngularJS commonly follows an MVC-based approach, while modern Angular uses a component-based architecture.

Component in Angular

Definition: A component is a basic building block of an Angular application. It controls a specific part of the user interface and combines the HTML template, CSS styles, and TypeScript code required to display and manage that part.

1. Component-Based Architecture: Angular applications are divided into small, reusable components. Each component manages a particular part of the application, such as a header, navigation bar, or login form.

2. TypeScript Class: The component class contains the logic and data required by the component. It is written in TypeScript.

3. HTML Template: The template defines the structure and content displayed to the user. It uses HTML and Angular template syntax.

4. CSS Styles: CSS styles control the appearance of the component, including colors, fonts, spacing, and layout.

5. Component Decorator: Angular uses the @Component decorator to identify a class as a component and provide its configuration, such as the selector, template, and styles.

6. Selector: The selector defines the custom HTML tag used to include a component in another template.

7. Reusability: Components can be reused in different parts of an application, reducing code duplication and making development easier.

8. Data Binding: Components use data binding to connect TypeScript data with the HTML template and display dynamic information.

Example:

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-hello',
  standalone: true,
  template: '<h1>Hello, Angular!</h1>'
})
export class HelloComponent {
}
```

Output : Hello, Angular!

Conclusion: A component is the fundamental building block of Angular applications. It combines a TypeScript class, an HTML template, and styles to create reusable and maintainable user interface elements.

Component-Based Model in Angular

Definition: The component-based model is an architectural approach in Angular in which an application is divided into small, independent, and reusable components. Each component manages a specific part of the user interface and its related logic.

1. Component-Based Architecture: An Angular application consists of multiple components, such as a header, sidebar, login form, and footer. These components work together to form a complete application.

2. Independent Components: Each component manages its own data, template, and behavior, making the application easier to organize and develop.

3. Reusability: Components can be reused in different parts of an application, reducing code duplication and development time.

4. HTML Template: Each component has a template that defines how its content is displayed to the user.

5. TypeScript Class: The component class contains the data, properties, and methods required to manage the component's behavior.

6. CSS Styling: Components can have their own styles to control their appearance and layout.

7. Communication Between Components: Angular supports communication between components using mechanisms such as input properties and output events.

8. Easy Maintenance: Since the application is divided into smaller components, developers can modify or debug individual components without unnecessarily changing the entire application.

9. Scalability: New components can be added as the application grows, making this approach suitable for large applications.

Example: In an online shopping application, the header, product list, product details, shopping cart, and footer can each be developed as separate Angular components.

Conclusion: The component-based model is the foundation of modern Angular applications. It improves reusability, organization, maintainability, and scalability by dividing an application into smaller components.

Setting up Angular

Definition: Setting up Angular means preparing the development environment required to create, run, and develop Angular applications. It involves installing Node.js, NPM, Angular CLI, and a suitable code editor.

1. Install Node.js: Node.js is a JavaScript runtime environment required to run development tools used by Angular. Download and install Node.js from the official website: https://nodejs.org/

2. Install NPM: NPM stands for Node Package Manager. It is included with Node.js and is used to install and manage the packages required for Angular development.

3. Check Node.js and NPM Installation: After installation, open the terminal or command prompt and execute the following commands:

```
node -v
npm -v
```

These commands display the installed versions of Node.js and NPM.

4. Install Angular CLI: Angular CLI stands for Angular Command-Line Interface. It provides commands to create, develop, test, and build Angular applications.

```
npm install -g @angular/cli
```

5. Verify Angular CLI: After installation, execute the following command to check whether Angular CLI is installed successfully.

```
ng version
```

6. Install a Code Editor: Install a suitable code editor, such as Visual Studio Code, to write and manage Angular application code.

7. Create an Angular Project: Use Angular CLI to create a new project.

```
ng new my-app
```

8. Run the Angular Application: Move into the project directory and start the development server.

```
cd my-app
ng serve --open
```

The application runs on the local development server, usually at http://localhost:4200/.

Conclusion: Setting up Angular involves installing Node.js, NPM, Angular CLI, and a code editor. Once the environment is ready, developers can create and run Angular applications efficiently.

Angular CLI

Definition: Angular CLI stands for Angular Command-Line Interface. It is a command-line tool used to create, develop, test, and build Angular applications. It simplifies Angular development by providing commands for common tasks.

1. Project Creation: Angular CLI allows developers to create a new Angular project using a single command.

```
ng new my-app
```

2. Development Server: It provides a development server to run an Angular application locally.

```
ng serve
```

3. Component Creation: Angular CLI can generate new components automatically.

```
ng generate component header
```

4. Service Creation: It can generate services to organize reusable business logic and shared functionality.

```
ng generate service user
```

5. Application Build: Angular CLI compiles and builds an Angular application for deployment.

```
ng build
```

6. Testing: It provides commands to run the tests configured for an Angular project.

```
ng test
```

7. Dependency Management: Angular CLI works with NPM to install and manage the packages required by an Angular project.

8. Code Generation: It automatically generates files and basic code structures for components, services, and other Angular features, reducing manual work.

9. Installation: Angular CLI can be installed globally using NPM.

```
npm install -g @angular/cli
```

10. Version Checking: The following command displays information about the installed Angular CLI and project environment.

```
ng version
```

Conclusion: Angular CLI is an important development tool that simplifies Angular project creation, code generation, testing, and building through simple commands.

Creating and Running an Angular Project

Definition: Creating and running an Angular project means generating a new Angular application using Angular CLI and executing it on a local development server. Angular CLI simplifies project setup by automatically creating the required files and folders.

1. Install Angular CLI: First, install Angular CLI globally using NPM.
npm install -g @angular/cli

2. Create a New Project: Use the ng new command to create a new Angular project named my-app.
ng new my-app

3. Configure the Project: During project creation, Angular CLI may ask questions about routing, stylesheets, and other project settings. Select the required options to complete the setup.
4. Open the Project Directory: Move into the newly created project folder using the following command.
cd my-app

5. Install Dependencies: Angular CLI installs the required dependencies during project creation. If necessary, dependencies can be installed manually using:
npm install
6. Run the Project: Start the Angular development server using the following command.
ng serve

7. Open the Application: Open a web browser and visit the following address:
http://localhost:4200/

The Angular application will be displayed in the browser. The default development server usually uses port 4200.

8. Run and Open Automatically: To start the server and open the application in a browser automatically, use:
ng serve --open

9. Modify the Application: Edit the component files in the project to change the application's content. The development server can automatically reload the application when supported files are changed.

10. Stop the Server: To stop the development server, press Ctrl + C in the terminal.

Conclusion: Angular CLI makes it easy to create and run Angular projects. Commands such as ng new, cd, and ng serve help developers set up, access, and execute an Angular application locally.

Dependencies in Angular

Definition: Dependencies in Angular are external libraries, packages, and modules required by an application to perform specific tasks. Angular uses NPM (Node Package Manager) to install and manage these dependencies.

1. Angular Dependencies: Angular dependencies are packages required to run an Angular application. Examples include @angular/core, @angular/common, and @angular/router.

2. Development Dependencies: Development dependencies are packages used during development, testing, and building but are not generally required as runtime packages in the deployed application. Examples include testing tools and build tools.

3. package.json File: The package.json file stores project information, scripts, dependencies, and development dependencies.

4. Installing Dependencies: Dependencies can be installed using the NPM install command

```
npm install
```

This command installs the packages listed in the project's package.json file.

5. Adding a Dependency: A new package can be added to a project using NPM.

```
npm install package-name
```

Replace package-name with the name of the required package. NPM normally adds the package to the dependencies section of package.json.

6. Development Dependency Installation: A package required only for development can be installed using the following command.

```
npm install package-name --save-dev
```

This adds the package to the devDependencies section of package.json.

7. node_modules Folder: The node_modules folder contains the installed packages required by the project. It is generally generated automatically by NPM.

8. package-lock.json File: The package-lock.json file records the exact dependency versions and dependency tree installed by NPM. It helps maintain consistent installations across different environments.

9. Updating Dependencies: Dependencies can be updated using NPM commands, subject to the version ranges and compatibility requirements defined by the project.

```
npm update
```

10. Importance of Dependencies: Dependencies reduce the need to write everything from scratch, provide reusable functionality, and simplify application development and maintenance.

Conclusion: Dependencies are essential for Angular application development. NPM manages these packages, while package.json, package-lock.json, and node_modules help organize and maintain them.

App Component in Angular

Definition: The App Component is the root component of an Angular application. It acts as the main component that contains or organizes other components and provides the starting point for the application's user interface.

1. Root Component: The App Component is the main component of an Angular application. Other components can be included within its template to build the complete application interface.

2. TypeScript Class: The App Component contains a TypeScript class that manages the component's data, properties, and methods.

3. HTML Template: The template defines the content displayed on the screen. It may contain HTML elements and other Angular components.

4. CSS Styles: The App Component can use CSS styles to control the appearance and layout of its user interface.

5. Component Decorator: The @Component decorator provides metadata such as the selector, template, and styles required to configure the component.

6. Selector: The selector identifies the component in an Angular template. In many Angular projects, the root component uses the selector app-root.

7. Bootstrapping: Angular starts the application by bootstrapping the root component. In modern standalone Angular applications, this is commonly configured in main.ts using bootstrapApplication().

8. Integration with Other Components: The App Component can display other components, allowing different parts of an application to work together.

Example:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  standalone: true,
  template: '<h1>Welcome to Angular</h1>'
})
export class AppComponent {
}
```

Output:
Welcome to Angular

Conclusion: The App Component is the root component of an Angular application. It provides the main user interface structure and serves as the starting point for organizing and displaying other components.

Anatomy of a Component in Angular

Definition: The anatomy of an Angular component refers to the main parts that make up a component. An Angular component generally consists of a TypeScript class, an HTML template, CSS styles, and the `@Component` decorator.

1. Component Decorator: The `@Component` decorator provides metadata that tells Angular how to configure and use the component. It is imported from `@angular/core`.

2. Selector: The selector defines the custom HTML tag used to include the component in another template. For example, `app-hello` can be used as a component selector.

3. TypeScript Class: The class contains the component's properties, data, and methods. It controls the component's behavior.

4. HTML Template: The template defines the structure and content displayed to the user. It can contain HTML elements and Angular template syntax.

5. CSS Styles: CSS styles define the appearance of the component, including colors, fonts, spacing, and layout.

6. Data Binding: Data binding connects the component's data with its template. It allows information to be displayed and user interactions to update data.

7. Methods and Properties: Properties store data, while methods define actions that the component can perform.

Example:

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-hello',
  standalone: true,
  template: '<h1>{{ message }}</h1>',
  styles: ['h1 { color: blue; }']
})
export class HelloComponent {
  message = 'Hello, Angular!';
}
```

Output:

Hello, Angular!

Conclusion: An Angular component combines metadata, a TypeScript class, an HTML template, and CSS styles to create a functional user interface element. These parts work together to control the component's appearance and behavior.

Creating Components in Angular

Definition: Creating components in Angular means generating separate user interface parts that perform specific tasks in an application. Components can be created manually or by using Angular CLI, which automatically generates the required files and code.

1. Using Angular CLI: Angular CLI provides the ng generate component command to create a new component easily.

```
ng generate component header
```

The short form of this command is:

```
ng g c header
```

2. Component Files: Angular CLI generally generates the following files for a component:

- `header.component.ts`: Contains the component class and metadata.
- `header.component.html`: Contains the HTML template.
- `header.component.css`: Contains the component's CSS styles.
- `header.component.spec.ts`: Contains the component's unit test setup.

The exact filenames and structure may vary depending on the Angular version and project configuration.

3. Component Class: The TypeScript file contains the class that manages the component's data and behavior.

4. HTML Template: The HTML file defines the content and structure displayed to the user.

5. CSS Styling: The CSS file controls the appearance of the component, including colors, fonts, and layout.

6. Component Decorator: The @Component decorator provides metadata such as the selector, template, and styles.

7. Using the Component: To display a component, include its selector in another component's template. In a standalone Angular application, the component must also be imported into the parent component.

Example:

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-header',
  standalone: true,
  template: '<h1>Welcome to My Website</h1>',
  styles: ['h1 { color: blue; }']
})
export class HeaderComponent {
}
```

Output:
Welcome to My Website

8. Importance of Components: Components divide an application into smaller, reusable parts. They improve code organization, simplify maintenance, and make application development easier.

Conclusion: Creating components is an important part of Angular development. Angular CLI simplifies the process by generating the required files and basic code, helping developers build organized and reusable applications.